

C# 10 bis C# 13



C# 10

<https://docs.microsoft.com/en-us/dotnet/csharp/whats-new/csharp-9>

C# 10 - Überblick

- Global usings
- Implicit usings
- File-scoped namespaces
- Record Structs
- Extended property patterns

Global usings – Default

- using Namespaces werden ausgelagert und global zentralisiert
- {Projektname}.GlobalUsings.g.cs bindet standardmäßig usings ein
- global usings = Projektweit

```
// <auto-generated/>
global using global::System;
global using global::System.Collections.Generic;
global using global::System.IO;
global using global::System.Linq;
global using global::System.Net.Http;
global using global::System.Threading;
global using global::System.Threading.Tasks;
```

```
Console.WriteLine("Hello World!"); //Kein using System; notwendig
```

Implizit Using

- Implizite Usings werden in der Projekt-Konfigurationsdatei definiert und aktiviert
- Mit Include und Remove könne die allgemeinen Globalen Usings manipuliert werden

```
GlobalUsingSample
1 // <auto-generated/>
2 global using global::System;
3 global using global::System.Collections.Generic;
4 global using global::System.IO;
5 global using global::System.Linq;
6 global using global::System.Net.Http;
7 global using global::System.Threading;
8 global using global::System.Threading.Tasks;
```

```
<PropertyGroup>
  <!-- Other properties like OutputType and TargetFramework -->
  <ImplicitUsings>enable</ImplicitUsings>
</PropertyGroup>
```

```
<ItemGroup>
  <Using Include="System.IO.Pipes" />
</ItemGroup>
```

```
<ItemGroup>
  <Using Remove="System.Threading.Tasks" />
</ItemGroup>
```

Global using – Erstellen

- Erstelle eine Usings.cs oder Import.cs
- {Projektname}.GlobalUsings.g.cs bindet per Default usings ein
- global usings gelten Projektweit
- Bei explizitem Gebrauch eines Namespaces kann man auch auf einfache using zugreifen

Usings.cs -o X

```
global using System.Diagnostics;
```

Imports.cs -o X

```
global using System.Text;
```

```
Stopwatch stopwatch = new Stopwatch();  
StringBuilder sb = new StringBuilder();
```

File Scoped Namespaces

- File Scoped Namespaces sind gegenüber Block-Namespaces eine Optimierung der Codezeilen
- Mehrere Namespaces nicht mehr möglich in einer Datei

<pre>namespace Sprachfeatures { static void Main(string[] args) { } }</pre>	<pre>namespace Sprachfeatures; static void Main(string[] args) { } }</pre>
---	---

Record Struct

- Kompaktere Schreibweise zu ausgeschriebenen Structs
- Selbe Funktionen, Eigenschaften, ... wie ein Record

```
public record struct Point3D(double X, double Y, double Z);
```


Erweitertes Property Pattern

```
string GetOrderProcessor(Order o)
{
    return o switch
    {
        { Bezahlung: { Preis: { Waehrung: "€" } } } => "EuroProcessor",
        { Kunde: { Typ: CustomerType.VIP }, Bezahlung: { Preis: { Waehrung: "$" } } } => "USDEURProcessor",
        { Bezahlung: { Bezahldatum: { Year: 2022 } } } => "LastYearProcessor"
    };
}

string GetOrderProcessorPlus(Order bestellung)
{
    return bestellung switch
    {
        { Bezahlung.Preis.Waehrung: "€" } => "EuroProcessor",
        { Kunde.Type: CustomerType.VIP, Bezahlung.Preis.Waehrung: "$" } => "USDEURProcessor",
        { Bezahlung.Bezahldatum.Year: 2022 } => "LastYearProcessor"
    };
}

public record Order(Payment Bezahlung, Customer Kunde);
public record Payment(Price Preis, DateTime Bezahldatum);
public record Customer(string Name, CustomerType Typ);
public record Price(decimal Menge, string Waehrung);
public enum CustomerType { New, Regular, VIP }
```

C# 11

Linq Order & OrderDescending

- Ermöglicht einfache Sortierung von einer Liste
- Nicht sonderlich nützlich bei Objektlisten

```
List<int> ints = Enumerable.Range(0, 50).ToList();
```

```
ints.OrderBy(i => i);
```

```
ints.OrderByDescending(i => i);
```

```
ints.Order();
```

```
ints.OrderByDescending();
```

Neue Typen

Typ	Name	Beschreibung	Größe
System.Half	Half	Gleitkommazahl	2 Byte
System.Int128	Int128	Ganzzahl	16 Byte
System.UInt128	UInt128	Ganzzahl	16 Byte
System.IntPtr	nint	Zeiger	4 oder 8 Byte
System.UIntPtr	nuint	Zeiger	4 oder 8 Byte

DateTime Micro-/Nanosecond

```
DateTime now = DateTime.Now;  
Console.WriteLine(now.Millisecond);  
Console.WriteLine(now.Nanosecond);  
Console.WriteLine(now.Ticks); //1 Nanosekunde = 100 Ticks
```

interpolated string – Zeilenumbrüche

```
int safetyScore = 50;

string message = $"The usage policy for {safetyScore} is {safetyScore switch
{
    > 90 => "Unlimited usage",
    > 80 => "General usage, with daily safety check",
    > 70 => "Issues must be addressed within 1 week",
    > 50 => "Issues must be addressed within 1 day",
    _ => "Issues must be addressed before continued use",
}}";
```

interpolated string – raw string literal

```
int X = 2;
int Y = 3;

var pointMessage = $""The point "{X}", {Y}" is {Math.Sqrt(X * X + Y * Y)} from the origin"";

Console.WriteLine(pointMessage);
// output: The point "2, 3" is 3.605551275463989 from the origin.
```

interpolated string – raw string literal

Es können mehrere `$`-Zeichen in einem interpolierten raw string literal verwendet werden, um `{` und `}`-Zeichen in die Ausgabezeichenfolge einzubetten

```
int X = 2;
int Y = 3;

var pointMessage = $$$"The point {{{X}}}, {{{Y}}} is {{Math.Sqrt(X * X + Y * Y)}} from the origin""";
Console.WriteLine(pointMessage);
// output: The point {2, 3} is 3.605551275463989 from the origin.
```


List Patterns

- The pattern [1, 2, .., 10] matches all of the following

```
int[] arr1 = { 1, 2, 10 };  
int[] arr2 = { 1, 2, 5, 10 };  
int[] arr3 = { 1, 2, 5, 6, 7, 8, 9, 10 };
```

List Patterns Beispiel 2

```
Console.WriteLine(CheckSwitch(new[] { 1, 2, 10 })); // prints 1
Console.WriteLine(CheckSwitch(new[] { 1, 2, 7, 3, 3, 10 })); // prints 1
Console.WriteLine(CheckSwitch(new[] { 1, 2 })); // prints 2
Console.WriteLine(CheckSwitch(new[] { 1, 3 })); // prints 3
Console.WriteLine(CheckSwitch(new[] { 1, 3, 5 })); // prints 4
Console.WriteLine(CheckSwitch(new[] { 2, 5, 6, 7 })); // prints 50

static int CheckSwitch(int[] values)
    => values switch
    {
        [1, 2, .., 10] => 1,
        [1, 2] => 2,
        [1, _] => 3,
        [1, ..] => 4,
        [..] => 50
    };
```

List Patterns Beispiel 3 – Slice Pattern

```
// Sample 3
static string CaptureSlice(int[] values)
=> values switch
{
    [1, .. var middle, _] => $"Middle {String.Join(separator: ", ", middle)}",
    [.. var all] => $"All {String.Join(separator: ", ", all)}"
};
```

Required-Keyword

```
public class Fahrzeug
{
    public required int ID;
    public required int MaxV { get; set; }
    public FahrzeugMarke Marke;
    [System.Diagnostics.CodeAnalysis.RequiredMembers]
    public Fahrzeug(int ID, int MaxV)
    {
        this.ID = ID;
        this.MaxV = MaxV;
    }
    public Fahrzeug() { }
}

public enum FahrzeugMarke { Audi, BMW, VW }
```

- Required erzwingt das Setzen bestimmter Felder/Properties
- Attribut auf Konstruktor muss gesetzt werden für die erste Initialisierung unten
- Seit C# 12 Attribut obsolet

```
new Fahrzeug(0, 300); //erlaubt mit einem speziellen Attribut
new Fahrzeug() { ID = 1, MaxV = 250 }; //erlaubt auch ohne Attribut
new Fahrzeug(); //nicht erlaubt, weil ID und MaxV nicht gesetzt
```



Fahrzeug.Fahrzeug() (+ 1 overload)

CS9003: Required member 'Fahrzeug.ID' must be set in the object initializer or attribute constructor.

CS9003: Required member 'Fahrzeug.MaxV' must be set in the object initializer or attribute constructor.

Show potential fixes (Alt+Enter or Ctrl+J)

Generic Attributes

- Vorher:

```
// Before C# 11:  
public class TypeAttribute : Attribute  
{  
    public TypeAttribute(Type t) => ParamType = t;  
  
    public Type ParamType { get; }  
}
```

- Verwenden des neuen Features:

```
// C# 11 feature:  
public class GenericAttribute<T> : Attribute { }
```

- Verwenden von typeof

```
[TypeAttribute(typeof(string))]  
public string Method() => default;
```

C# 12

Collection Expressions

```
int[] a = [1, 2, 3, 4, 5, 6, 7, 8];
```

```
List<string> b = ["Eins", "Zwei", "Drei"];
```

```
Dictionary<string, int> c = [{"Eins", 1}, {"Zwei", 2}, {"Drei", 3}];
```

```
int[,] twoD = [[1, 2, 3], [4, 5, 6], [7, 8, 9]];
```

```
List<string> str = [];
```

```
Dictionary<string, int> dict = [];
```

Alias any type

```
using Point = (int X, int Y);  
  
using Point3D = (int X, int Y, int Z);  
  
using IntList = List<int>;  
  
Point point = (10, 10);  
  
Point3D point3D = (10, 10);  
  
IntList intList = [];
```


Primary constructors

```
public class Point(double X, double Y)
{
    public double Distance(Point p2)
    {
        return Math.Sqrt(Math.Pow((X - Y), 2) + Math.Pow(p2.X - p2.Y, 2));
    }

    //Weitere Konstruktoren müssen mit this erstellt werden
    public Point() : this(0, 0) { }
}
```

Interceptors

```
1 Test("Hallo");
2
3 public void Print(string str) => Console.WriteLine(str);
4
5 public static class Interceptors
6 {
7     [InterceptsLocation("Program.cs", 1, 1)]
8     public static void InterceptorMethod(this Test t)
9     {
10         Console.WriteLine("Hallo Interceptor");
11     }
12 }
13
14 [AttributeUsage(AttributeTargets.Method)]
15 public sealed class InterceptsLocationAttribute
16     (string filePath, int line, int character) : Attribute { }
```

Inline arrays

```
[InlineArray(10)]
public struct Buffer
{
    private int _element0; //Gibt den Typen des Arrays vor
}

Buffer buffer = new Buffer();
for (int i = 0; i < 10; i++)
{
    buffer[i] = i;
}

foreach (int i in buffer)
{
    Console.WriteLine(i);
}
```

C# 13

params

```
int Summe(params int[] zahlen)
{
    return zahlen.Sum();
}
```

```
int Summe(params List<int> zahlen)
{
    return zahlen.Sum();
}
```

Index

```
var countdown = new TimerRemaining()
{
    buffer =
    {
        [^1] = 0,
        [^2] = 1,
        [^3] = 2,
        [^4] = 3,
        [^5] = 4
    }
};

public class TimerRemaining
{
    public int[] buffer { get; set; } = new int[10];
}
```

partial

```
public partial class Test
{
    public partial string Name { get; set; }
}
```

```
public partial class Test
{
    private string _name;

    public partial string Name
    {
        get => _name;
        set => _name = value;
    }
}
```

Überladungen priorisieren

```
//Priorität 0 (niedriger)  
[OverloadResolutionPriority(0)]  
public void Test(int x) { ... }
```

```
//Priorität 1 (höher)  
[OverloadResolutionPriority(1)]  
public void Test(double x) { ... }
```